Mind the Gap: Predicting Missing Water Meter Data using ML

Interview Preparation Guide

Newcastle University - School of Engineering
Duration: 60 hours (5 hours/week, March-May 2026)
Supervisor: Dr Oktay Cetinkaya | Industry Partner: Connexin

Part 1: Behavioral Questions (25)

Data Analysis & Research (5 questions)

1. Describe using data analysis to solve a problem. What insights did you discover?
2. Experience with incomplete/messy data?
3. Walk through a research project methodology
4. Time you challenged an assumption in data analysis
5. Exploratory analysis that led to unexpected findings

Problem-Solving (5 questions)

6. Choosing between multiple technical approaches
7. Debugging process for unexpected results
8. Optimizing code/system performance
9. Making trade-off decisions (accuracy vs speed, complexity vs maintainability)
10. Failed approach - how did you recognize and pivot?

Learning & Adaptability (5 questions)

11. Machine learning exposure and learning approach
12. Quickly learning new tools/frameworks
13. Project outside your comfort zone
14. Self-directed learning example
15. Applying theory to practical problems

Communication (5 questions)

16. Explaining complex concepts to non-technical people
17. Approach to industrial collaboration with Connexin
18. Receiving and responding to critical feedback

19. Documentation practices
20. Presenting technical work

Time Management (5 questions)

21. Planning for 6-week structured timeline
22. Balancing 5hrs/week with coursework
23. Prioritizing multiple deadlines
24. Iterative development approach
25. Handling unexpected obstacles

Part 2: Technical Questions (30)

A: Python & Data Processing (8)

Q1: Initial Data Quality

```python
df = pd.read_csv('meter_data.csv')
# Some readings are negative, timestamps duplicated
# Write code to identify and fix these issues
```

Q2: Detecting Missing Timestamps

```python
df = pd.DataFrame({
    'timestamp': ['2024-01-01 00:00', '2024-01-01 01:00',
                  '2024-01-01 03:00'],  # 02:00 missing
    'reading': [100, 105, 115]
})
# How to detect hour 02:00 is completely missing?
```

Q3: Feature Engineering What features would you create from timestamp and historical readings? Implement 3 features.

Q4: Train-Test Split

```python
X_train, X_test = train_test_split(X, y, shuffle=True)
# What's wrong for time series? How to fix?
```

Q5: Memory Efficiency

```python
```

```
df = pd.read_csv('10M_rows.csv')  # Might crash
# How to process efficiently?
```

Q6: Outliers vs Real Spikes How distinguish genuine consumption spike (pool filling) from sensor error?

Q7: Data Validation Write validation logic for water meter readings - what physical constraints?

Q8: Rolling Window with Gaps

```
python

df['ma'] = df['reading'].rolling(24).mean()
# Problem with missing data? Fix?
```

B: Machine Learning (10)

Q9: Baseline Models Design 3 simple baselines (e.g., linear interpolation). How evaluate?

Q10: LSTM vs RNN vs Transformer When use each? Computational trade-offs? Which start with?

Q11: Sequence Length How determine optimal lookback window? What factors?

Q12: LSTM Architecture

```
python

model = Sequential([
    layers.LSTM(64, input_shape=(24, 1)),
    layers.Dense(1)
])
# Too simple. What improvements?
```

Q13: Time Series Validation

```
python

history = model.fit(X_train, y_train, validation_split=0.2)
# What's wrong? Proper strategy?
```

Q14: Evaluation Metrics MSE alone insufficient. What other metrics? Why each important?

Q15: Overfitting Training loss 0.05, validation loss 0.35 and increasing. What's happening? Solutions?

Q16: Autoencoder Use How use autoencoder for anomaly detection and gap filling?

Q17: Seasonality Daily and weekly consumption patterns - how ensure model captures them?

Q18: Cold Start New meter, only 1 week data. How predict missing readings?

C: IoT & Wireless Networks (6)

Q19: LoRaWAN Packet Loss Why only 60-70% DSR vs 85% required? List 5+ causes.

Q20: Improving DSR 4 strategies to improve 70%→85% DSR without new gateways?

Q21: IoT Transmission Code

```c
void loop() {
    int reading = readMeter();
    if (hourPassed()) loraSend(reading);
    sleep(3600);
}
// What's missing for production?
```

Q22: Packet Design Design LoRaWAN packet (51 byte limit) for hourly readings.

Q23: Battery Life 2000mAh battery:

- Sleep: 0.01mA
- Reading: 5mA for 1s
- TX: 120mA for 2s
- Frequency: hourly Calculate battery life.

Q24: Edge Processing Strategy to transmit only "unpredictable" readings. Bandwidth reduction?

D: System Design (6)

Q25: End-to-End Architecture Design: [Meter] → [Gateway] → [Server] → [DB] → [ML] → [Dashboard] Key responsibilities and failure handling for each.

Q26: Real-Time vs Batch Should prediction be real-time or batch? Trade-offs?

Q27: Model Deployment LSTM achieves 90% accuracy. How deploy to production? Monitoring?

Q28: Multiple Meters Thousands of meters - one model or many? Handle different patterns?

Q29: Data Pipeline Design pipeline: ingestion → training → deployment. Tools?

Q30: Continuous Learning Patterns change over time. How detect degradation, retrain, A/B test, rollback?

Part 3: Sample Answers

Behavioral Answers

Q1: Data-Driven Problem Solving "In second year, sorting algorithm benchmarks were inconsistent. Instead of averaging, I collected 1000 measurements across different inputs. Box plots revealed Quicksort had outliers on sorted data due to pivot selection.

I modified to use randomized pivots—outliers disappeared. Learned that investigating anomalies reveals systemic issues. Created visualization showing before/after distributions in my report."

Why strong: Specific, shows initiative, methodical, actionable solution, learning.

Q11: Machine Learning "Started with Andrew Ng's Coursera (10hrs/week, 8 weeks). Math was abstract until I implemented from scratch—watched gradient descent update weights and loss decrease.

Built progression: linear regression (housing), logistic regression (spam), neural net (MNIST - 94% accuracy). Learned ML is optimization—searching function space. Data quality beats model complexity.

Currently learning RNNs/time series via fast.ai—directly relevant to internship."

Why strong: Self-motivated, concrete path, hands-on, depth of understanding, connects to role.

Q17: Industrial Collaboration "Would adapt to Connexin's priorities—practical results, timelines, business impact.

Weekly updates:

- Accomplished (e.g., 'baseline 80% accurate')
- Blockers
- Next steps
- Interesting findings

Avoid jargon. Instead of 'LSTM with 128 units, 0.2 dropout' say 'neural network predicting 87% accurately, 7% better than interpolation.'

Final presentation:

1. Problem (70% DSR vs 85%, densification cost)
2. Approach
3. Results (clear metrics, visuals)
4. Recommendations (which solution, why)
5. Deployment (cost, timeline, integration)

Prepare for: cost, timeline, reliability, edge cases questions.

Seek their domain expertise—they know water infrastructure better. Two-way communication."

Why strong: Understands priorities, concrete strategies, structured presentation, anticipates questions, values their input.

Technical Answers

Q2: Missing Timestamps

Simple `.isna()` only catches nulls in existing rows, not missing rows!

```python
```

```python
def detect_gaps(df, freq='H'):
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df = df.sort_values('timestamp')

    # Create expected range
    full_range = pd.date_range(
        start=df['timestamp'].min(),
        end=df['timestamp'].max(),
        freq=freq
    )

    # Find missing
    existing = set(df['timestamp'])
    missing = [t for t in full_range if t not in existing]

    return missing

# Better: use reindex
df = df.set_index('timestamp')
full_idx = pd.date_range(start=df.index.min(),
                         end=df.index.max(), freq='H')
df_complete = df.reindex(full_idx)
gaps = df_complete[df_complete.isna().any(axis=1)]
```

Q4: Train-Test Split

Problem: `shuffle=True` breaks temporal order. Model trains on future, tests on past!

```python
# WRONG
X_train, X_test = train_test_split(X, y, shuffle=True)

# CORRECT - Chronological
df = df.sort_values('timestamp')
split_idx = int(len(df) * 0.8)
train = df.iloc[:split_idx]
test = df.iloc[split_idx:]

# BETTER - Walk-forward validation
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
for train_idx, test_idx in tscv.split(X):
    # Training set grows each fold
    pass
```

Why: In production, predict future from past. Random split is unrealistic.

Q9: Baseline Models

1. Linear Interpolation

```python
filled = series.interpolate(method='linear')
# Good for short gaps, fast
```

2. Seasonal Average

```python
df['hour'] = df['timestamp'].dt.hour
df['dow'] = df['timestamp'].dt.dayofweek
seasonal = df.groupby(['hour', 'dow'])['reading'].transform('mean')
df['filled'] = df['reading'].fillna(seasonal)
# Captures daily/weekly patterns
```

3. Previous Week Same Time

```python
filled = series.fillna(series.shift(168))  # 168 hours = 1 week
# Good for strong weekly patterns
```

Evaluation:

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Create synthetic gaps (15% removal)
test_df = df.copy()
mask = test_df.sample(frac=0.15).index
true_values = test_df.loc[mask, 'reading']
test_df.loc[mask, 'reading'] = np.nan

# Test each baseline
for name, method in baselines:
    pred = method(test_df)
    rmse = sqrt(mean_squared_error(true_values, pred.loc[mask]))
    mae = mean_absolute_error(true_values, pred.loc[mask])
    print(f"{name}: RMSE={rmse:.2f}, MAE={mae:.2f}")
```

Use best baseline as minimum threshold. ML must beat by >15%.

Q12: LSTM Improvements

Basic model too simple:

```python
model = Sequential([
    # Stack multiple layers for hierarchical patterns
    layers.LSTM(64, return_sequences=True,
                input_shape=(seq_len, n_features),
                dropout=0.2, recurrent_dropout=0.2),
    layers.LSTM(32, dropout=0.2),

    # Batch normalization
    layers.BatchNormalization(),

    # Dense with regularization
    layers.Dense(16, activation='relu',
                 kernel_regularizer=regularizers.l2(0.01)),
    layers.Dropout(0.3),
    layers.Dense(1)
])

# Callbacks
early_stop = EarlyStopping(monitor='val_loss', patience=10,
                           restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                              patience=5)

model.fit(X_train, y_train, validation_split=0.2,
          epochs=100, callbacks=[early_stop, reduce_lr])
```

Improvements:

- Multiple layers: short + long-term patterns
- Dropout 0.2-0.3: prevent overfitting
- Batch norm: training stability
- L2 regularization: simpler models
- Multiple features (not just readings): hour, day, rolling avg
- Early stopping: don't overfit
- Learning rate scheduling: better convergence

Q19: LoRaWAN Packet Loss Causes

10 Main Causes:

1. **Distance/Path Loss:** Signal decreases with distance (inverse square law). Urban 2-5km, Rural 10-15km.
2. **Physical Obstacles:** Buildings (10-20dB), concrete (10-15dB), underground installation (severe), metal objects.
3. **Network Congestion:** Multiple devices transmitting simultaneously. Duty cycle limits (1% EU). Limited gateway channels (8). ALOHA protocol = collisions.
4. **Interference:** Other IoT devices, industrial equipment, WiFi, power line harmonics.
5. **Weather:** Rain increases attenuation, temperature affects battery, humidity causes condensation.
6. **Gateway Limits:** Processing capacity, backhaul issues, positioning compromises.
7. **Spreading Factor Trade-offs:** SF12 takes 128x longer than SF7 → more collision probability.
8. **Battery Degradation:** Weak battery → lower TX power, temperature effects.
9. **Time Sync Issues:** Clock drift, timestamp misalignment, join overhead.
10. **Installation Quality:** Poor antenna placement, water ingress, grounding issues.

Result: Typical urban 60-70% vs required 85% = 15-25% improvement needed.

Q20: Improving DSR (70%→85%)

4 Key Strategies:

1. Adaptive Data Rate (ADR) - +5-8%

- Strong signal: SF7, low power → fast TX, less collisions
- Weak signal: SF12, max power → better range
- Meters adapt based on recent RSSI history

2. Smart Retry + Backoff - +8-12%

- Current: send once, if fail, lost
- Improved: retry 3x with delays (10s, 30s, 60s) + jitter
- Buffer failed transmissions

3. Window Randomization - +4-6%

- Current: all meters TX at top of hour → collision spike
- Improved: spread over 5-10 min window, random offset per meter

4. Data Buffering/Batching - +5-7%

- Buffer 24hrs of readings

- Send multiple per packet
- One success = multiple readings delivered

Combined: 70% + 20-25% = **90% DSR** (exceeds 85% target)

Trade-offs:

- More retries = higher battery use (manageable)
- Buffering = delayed reporting (acceptable)
- No infrastructure cost - firmware updates only

Q23: Battery Life

Given:

- 2000mAh battery
- Sleep: 0.01mA
- Reading: 5mA × 1s
- TX: 120mA × 2s
- Frequency: hourly

Per hour:

```
Sleep: 0.01mA × 3598s ≈ 0.01mAh
Reading: 5mA × (1/3600)h ≈ 0.0014mAh
TX: 120mA × (2/3600)h ≈ 0.067mAh
Total: 0.078mAh/hour
```

Per day: 24 × 0.078 = 1.87mAh

Battery life: 2000 / 1.87 = **1,070 days = 2.9 years**

Problem: Need 5+ years!

Solutions:

1. **TX every 2 hours:** 2.9 × 2 = 5.8 years ✓
2. **Optimize TX (ADR):** 90mA × 1.5s → 4.6 years
3. **Deep sleep:** 0.005mA → 3.3 years
4. **Combined:** 9.3 years theoretical, **6-7 years realistic** ✓✓

Key: TX dominates power. Reducing frequency/optimizing has biggest impact.

Q25: System Architecture

Components & Responsibilities:

1. Water Meter

- Pulse counting, buffering (24-48hrs), timestamping, TX
- Tech: STM32L (low power), SX1276 (LoRa), RTC, flash
- Failure: buffer if TX fails, retry with backoff, emergency beacon

2. LoRaWAN Gateway

- Receive from meters, forward via backhaul
- Tech: SX1301 concentrator, Ethernet/4G, GPS sync
- Failure: queue if backhaul down, battery backup, watchdog

3. Network Server

- Authentication, deduplication, routing, ADR management
- Tech: ChirpStack / AWS IoT Core, device registry DB
- Failure: redundant servers, persistent queue (Kafka), alerts

4. Database (Time Series)

- Store raw + gap-filled data
- Tech: InfluxDB / TimescaleDB
- Failure: replication, backups, WAL

5. Gap Detection Service

- Monitor for gaps, categorize, trigger ML pipeline
- Tech: Python microservice, Airflow scheduling
- Failure: idempotent processing, dead letter queue

6. ML Prediction Pipeline

- Feature engineering, run models, generate predictions
- Tech: TensorFlow/PyTorch, Docker, TF Serving
- Failure: fallback to baseline, log predictions, A/B testing

7. Validation Service

- Check predictions physically reasonable, flag low confidence
- Tech: Python with business rules
- Failure: conservative (reject if uncertain), manual review

8. API Layer

- Serve data, auth, rate limiting, caching
- Tech: FastAPI, Redis cache, API gateway
- Failure: return cached data, circuit breaker, retries

9. Dashboard

- Real-time status, DSR metrics, gap visualization
- Tech: Grafana / React, WebSocket
- Failure: client caching, show last known state

10. Monitoring

- System health, model performance, alerts
- Tech: Prometheus, Grafana, PagerDuty
- Failure: redundant monitoring

Data Flow: Meter → Gateway → Network Server → DB → Gap Detection → ML Pipeline → Validation → API → Dashboard

Scalability: Thousands of meters = batch processing, sharded DB. Millions = Kafka streaming, Spark ML.

Q30: Continuous Learning

Challenge: Patterns change (seasonal, lifestyle, appliances, leaks).

Solution Design:

1. Performance Monitoring

```python
def monitor_predictions(predictions, actuals):
    mae = mean_absolute_error(actuals, predictions)
    degradation = (mae - baseline_mae) / baseline_mae

    if degradation > 0.15:  # 15% worse
        trigger_retraining_alert()

    log_metric('model_mae', mae)
```

2. Automated Retraining

- **Schedule:** Weekly (Sunday 2AM) OR on degradation alert
- **Data:** Last 90 days (rolling window)
- **Process:** Fetch data → preprocess → train → validate → compare → stage for A/B

3. A/B Testing

```python
def predict(meter_id, features):
    if hash(meter_id) % 10 == 0:  # 10% traffic
        return model_B.predict(features)  # New model
    else:
        return model_A.predict(features)  # Current
```

- Day 1-3: 10% on new model
- Day 4-7: If good, 25%
- Day 8-14: 50%
- Day 15+: 100%

4. Gradual Rollout Monitor at each stage: accuracy, latency, errors, complaints

5. Rollback Strategy

```python
if current_mae > previous_mae * 1.2:  # 20% worse
    rollback_to_previous_version()
    alert_team()
```

6. Seasonal Models

- Winter model (Nov-Feb): higher baseline
- Summer model (Jun-Aug): different pattern
- Auto-switch by calendar

7. Data Drift Detection

```python
from scipy.stats import ks_2samp
for feature in features:
    stat, p = ks_2samp(baseline_data[feature], current_data[feature])
    if p < 0.05:  # Significant change
        trigger_retraining()
```

8. Metrics Dashboard Real-time: current version, MAE trends, A/B results, retraining schedule

Timeline: Week 1-2: performs well → Week 3: degradation → Week 4: auto-retrain → Week 5: A/B test → Week 6: gradual rollout → Week 7: 100%, restored

Principles:

- Monitor continuously

- Automate with human oversight
- Never deploy to 100% immediately
- Easy rollback
- Track everything

Key Preparation Areas

1. **Python/Pandas:** Time series, data manipulation, memory efficiency
2. **Machine Learning:** LSTM, evaluation metrics, overfitting, validation
3. **IoT:** LoRaWAN basics, packet loss, power management
4. **System Design:** End-to-end architecture, scalability, failure handling
5. **Statistics:** Time series methods, outlier detection

Resources

- Pandas time series documentation
- TensorFlow LSTM tutorials
- LoRaWAN specification
- Time series forecasting (Prophet, ARIMA concepts)
- Model evaluation metrics

Interview Tips

Behavioral:

- Use STAR method
- Be specific with examples
- Show learning mindset
- Connect to internship

Technical:

- Think out loud
- Consider edge cases
- Discuss trade-offs
- Ask clarifying questions
- If unsure, explain how you'd investigate

Good luck!